These rules cover most access scenarios. Referring to the graph in figure 6.1:

- Alice, a solution architect, has full access to and can share the *deployment plan* she owns.
- Bob, a support engineer, can change *support ticket #456* because he's the ticket's editor.
- Carol can delete *support ticket #456* because she's the owner.
- Carol, Alice's manager, can access the *deployment plan* because *Alice owns it*—a transitive rule that depends on the static policy and dynamic graph.

This final example is worth noting. Carol's access depends on the chain of manager → owner → resource. If Alice is removed as the owner of the resource, Carol's access will also disappear. There's no need to explicitly revoke permissions.

Rules that `permit all` are not limited to viewing or editing. In ACME's system, `all` includes:

- Updating or deleting the resource
- Granting or revoking others' access
- Assigning ownership

As I've mentioned, the rules remain fixed: they don't change daily. What changes is the data—the graph of relationships that the system evaluates against. *That graph is the policy.* Whether access is granted depends not on rewriting the logic but on updating the relationships. If Carol is assigned as a manager of the Customer Success team, she gains access to what her team owns. If she's removed from that relationship, she loses it. This dynamic assessment, based on a static set of rules, enables fine-grained, flexible access decisions without overly complicating the authorization logic.

This approach offers remarkable flexibility because it makes controlling access simple. Anyone with the right permissions can shape policy just by managing relationships. Assigning a new owner to a document, giving a team editing rights, or designating a customer contact as a collaborator—all these actions become policy updates. There's no need to write or deploy new